



Film Shot Type Classification Based on Camera Movement Styles

Antonia Petrogianni¹, Panagiotis Koromilas²(✉) ,
and Theodoros Giannakopoulos²

¹ University of Thessaly, Volos, Greece
apetrogianni@uth.gr

² National Center for Scientific Research - Demokritos, Athens, Greece
{pakoromilas,tyianak}@iit.demokritos.gr

Abstract. Visual information contains the most important characteristics of a movie regarding the related content and filming techniques. Especially the way the camera moves to capture the scene is vital to define the director's aesthetics. However, most of the machine learning tasks existing in the literature treat the movie as shallow content, rather than as an artistic work, and therefore focus on detecting objects and faces, recognizing activities and extracting plot-related topics. On the other hand, cinematography is closely connected to the choice of different ways to handle the camera, and thus camera movements include information that is useful in order to analyse the artistic style of a movie. In this work we present an original, publicly available (https://github.com/magcil/movie_shot_classification_dataset) dataset for film shot type classification that is associated with the distinction across 10 types of camera movements that cover the vast majority of types of shots in real movies. In addition, two different methods are evaluated on the new dataset, one static that is based on feature statistics across frames, and one sequential that tries to predict the target class based on the input frame sequence using LSTMs. Based on the evaluation process it is inferred that the sequential method is more suited for modeling the camera movements.

Keywords: Shot classification · Camera movement classification · Movie analysis

1 Introduction

Machine learning methodologies have been greatly applied in order to conduct movie analysis. Extensive research has been published on different kinds of problems such as movie recommendation [8, 9, 16, 29], movie summarization [25, 30, 31], movie genre classification [10, 27, 34] or movie scene detection [2, 12, 24]. These problems have been faced using different types of approaches (e.g. traditional machine learning or deep learning architectures) or different types of representations (e.g. multimodal recommendation [5] or summarization [22]).

Most existing tasks in computer vision primarily focus on content rather than style understanding. That is the analysis of movies, which are a work of art, cannot be properly conducted when excluding cinematographic techniques used by the director. Shot type classification could enrich movie analysis techniques and result in a more inclusive automated movie understanding.

The literature on shot classification is mostly centered around machine learning methods in sports events (e.g. [20]) that classify the corresponding shots into one of the following basic categories: long, medium, close and out-field respectively. Most of the movie shot analysis methods adopt this type of categorization in order to form the shot scale classification problem, where a shot is classified based on the apparent distance of the camera from the main subject of a scene. Most of the works (e.g. [3, 7]) examine the three-class problem of long, medium and close-up shots. Recent methods have employed Deep Learning architectures [26] and others enrich their overall architecture with the use of semantic segmentation [1]. However, in these methods, the shot analysis is on frame level rather than on video level, and thus the Deep Learning architecture does not include the temporal dimension.

One of the first works to introduce a cinematographic shot taxonomy based on camera movements is [33], which ends up with the following classes: stationary, contextual-tracking, focus-tracking, focus-in, focus-out, intermittent/planing establishment and chaotic. This taxonomy is followed by [13] where a new dataset of 5226 shots is created. A method for videography-based video analysis is introduced in [17], where camera operation classification is used as a module on the proposed architecture in order to classify the shot among four categories, namely static, pan, tilt and zoom.

The so far mentioned works on camera movement classification and their predecessors (e.g. [21]) evaluate their respective approaches on their own private collections, which are not made available. The first publicly available dataset is introduced in [4] with shots categorized as aerial, bird eye, crane, dolly, establishing, pan, tilt, and zoom. However this collection is rather small, consisting of just 263 shots, where most of them are not from movies. The most relevant work is that of [23] MovieShot, a publicly available dataset that contains classes for both scale and camera movement. Still, the proposed camera movement types are rather generic including only static, motion, push shot (zoom in) and pull shot (zoom out).

To sum up, the existing datasets are either private or do not focus on camera movements, where some of them are not movie-oriented since they include other types of videos. However, camera movement is important for expressing mood and style in a movie, and as such it is crucial to be considered as an attribute in any movie analysis system: movie indexing and search, movie visualization and, of course, movie recommendation systems. To put it simple, the way the directors decide to move their cameras makes us, the viewers, like or not a movie, among other characteristics. In this work, we propose a publicly available dataset that contains shots coming strictly from movies. The proposed collection is concerned about camera movements and proposes the categorization of shots

in the following extensive 10 classes: static, panoramic, zoom-in, travelling-out, vertical movement, aerial, travelling-in, tilt, handheld, panoramic lateral. We report classification metrics on our dataset based on (i) a static method which is based on aggregated statistics on the feature sequence, and (ii) a sequential method where an LSTM is applied directly on the sequential features which is not the usual case in the literature since most of the existing works depend on feature aggregations [23].

The rest of the paper is organized as follows. In Sect. 2 the dataset compilation along with the annotation procedure is discussed. The feature extraction process, as well as the proposed methodologies are explained in Sect. 3. Experimental results are provided in Sect. 4, where Sect. 5 concludes our work and proposes some future work directions.

2 Dataset Compilation

2.1 Video Shot Generation

Video *shots* are basic structural elements in film-making and video production that contain a series of frames and run for an uninterrupted period of time [6]. Types of shots can be characterized (among others) with regards to the respective camera movement. Camera movement in film-making is a technique that causes a change in frame or perspective through the movement of the camera. The types of camera movement are crucial in the direction film process, since through these, directors can cause separate feelings to the audience: for example, fast camera movements can cause the viewer anxiety or irritation.

In this work, our goal is to classify the types of shots based on the cinematic aesthetics of camera movements. Towards this end, we have created a manually annotated dataset. To our best knowledge, there is no corresponding dataset that represents a wide range of camera movement styles that is able to cover the vast majority of shot types in any movie. The dataset is created using video shots from 48 films in which a basic shot detection algorithm was applied to generate successive shots. The generated detected video shots were randomly sampled and then led to a multi-annotator pipeline, leaning to the final annotated dataset.

In order to detect shots from movies, we adopted a very basic and fast shot-detection algorithm that is based on three thresholding criteria. In particular, as a first step, the following features are computed on a frame basis:

- Average value of magnitudes of optical flow vectors (*mag_mu*). Optical flow vectors are modeling local movements of video blocks in successive frames [19]. In other words, a flow vector indicates how a particular block from a frame will “move” into the next frame. The rationale behind using this feature is that, if the value of magnitudes of flow vectors changed abruptly from the current frame to the next, then it would probably occur a shot change.
- Proportion of pixels with high absolute differences between two successive frames (*gray_diff*): in particular, we count pixels whose differences is over 50 between two successive frames.

- Average absolute diffs in the histograms of the gray values between two successive video frames (f_diff).

In order to select the aforementioned parameters, we created a small dataset of manually annotated shots (in terms of segment endpoints) and selected the parameters values $mag_mu = 0.08$, $gray_diff = 0.65$ and $f_diff = 0.02$ which resulted to an almost 80 macro F1 performance in the binary task of endpoint shot detection, with a tolerance of 1 s (i.e. the allowed time distance from the ground truth shot endpoints). The aforementioned method has been implemented in the GitHub repository that also implements the basic feature extraction adopted in this work, called *multimodal_movie_analysis*¹.

As soon as this parameter setting of the shot detection algorithm was completed, we executed the algorithm on 48 movies from various genres and eras. This process resulted in around 80,000 detected shots. We then discarded shots shorter than 2 s and randomly selected 4000 shots as the final “annotation poo”. In the next Sections we describe the annotation process of these 4000 shots and the way we handled the inter-annotator agreement to form the final ground-truth.

2.2 Video Shot Taxonomy

In order to define the set of classes in which the shots have been labelled, we collaborated with a professional director and at the same time we focused in having a minimum set of classes that covers as many shots as possible from all movies. Our basic criterion for defining the classes was the *type of camera movement*. Based on this, the following classes have been defined:

1. **Static**, The camera is locked on a tripod or pedestal and remains still. Among other types of scenes, it is commonly used in dialogues. A static camera does not necessarily indicate a static scene. Actors and even the background can move while the camera remains still. ([Link-for-static-video](#))
2. **Vertical movement**, of the camera lens while the camera remains locked on a tripod. It is equivalent to someone tilting his head up/down. ([Link-for-vertical-movement-video](#))
3. **Tilt**, Moving the entire camera up or down without moving its lens. Tilting up is like one is moving up his entire body from a sitting position. ([Link-for-Tilt-video](#))
4. **Panoramic**, Lateral movement of the camera lens while the camera remains locked down on its tripod or pedestal. It is like someone is moving his head from one side to another. ([Link-for-Panoramic-video](#))
5. **Panoramic Lateral**, The camera follows the action moving parallel to characters. Specifically, the camera captures the lateral movement of the subject, for example the camera moves parallel to a person walking down the street to keep them in the frame. ([Link-for-Panoramic-lateral-video](#))

¹ https://github.com/tyiannak/multimodal_movie_analysis.

6. **Travelling in**, in this type of shot, the camera moves forward, pushes in a character or follows a character. ([Link-for-Travelling-in-video](#))
7. **Travelling out**, in this type of shot, the camera pulls out, moving away from the subject and revealing the surroundings. ([Link-for-Travelling-out-video](#))
8. **Zoom in**, In this type of shot, the camera lens are adjusted so that the image gradually appears larger and closer. ([Link-for-Zoom-in-video](#))
9. **Aerial**, the camera is flown above the action, using a helicopter, drone or a plane. ([Link-for-Aerial-video](#))
10. **Handheld**, the camera is moving throughout the filming set, while the camera operator is physically holding it. These camera shots are shaky and create a hectic feeling. ([Link-for-Handheld-video](#))

Initially we used five more classes that have been found in the literature, but these have been rarely found in the annotation process (in particular they gathered less than 30 annotations after the aggregation procedure described in the next Section). These classes are: (1) **Car Front Windshield**, the camera is mounted on the front windshield, (2) **Car Side Mirror**, the camera is mounted on the side mirror and the viewer can see the driver (and co-driver) from the side (3) **Zoom out**, the entire image appears much smaller and further away (4) **Vertigo**, a combination of travelling and zoom and (5) **Panoramic 360**, a semicircular movement of the camera. These types of shots appeared in less than 30 annotations (i.e. in less than 0.75% of the total data), and therefore their recognition would be of low significance in a real-world scenario of movie analysis. Furthermore, any supervised procedure would fail to model these classes with such few data points.

2.3 Annotation Process

As soon as the 4000 shots had been selected from the 48 films, we began the annotation procedure. Towards this end, we used a simple Python-based video annotation web tool². Figure 1(a) presents a screenshot of the tool: the user can simply view the video and assign it a video shot label. Note that the users had also the ability to annotate a video as “N/A”: this label was used for cases that corresponded to either corrupted videos, or to videos with a non clear and fuzzy type of camera movement, or even to videos with more than one types of camera movements. Video shots that have been annotated as “N/A” were discarded from the final dataset.

This process has been carried out by 17 human annotators that annotated an arbitrary number of randomly selected shots. In total, 7500 individual annotations have been made. Then we proceeded to applying a simple aggregation rule: *for each video shot, two minimum annotations (by two different annotators) were required, along with a minimum annotator agreement of 60%.*

² https://github.com/theopsall/video_annotator.

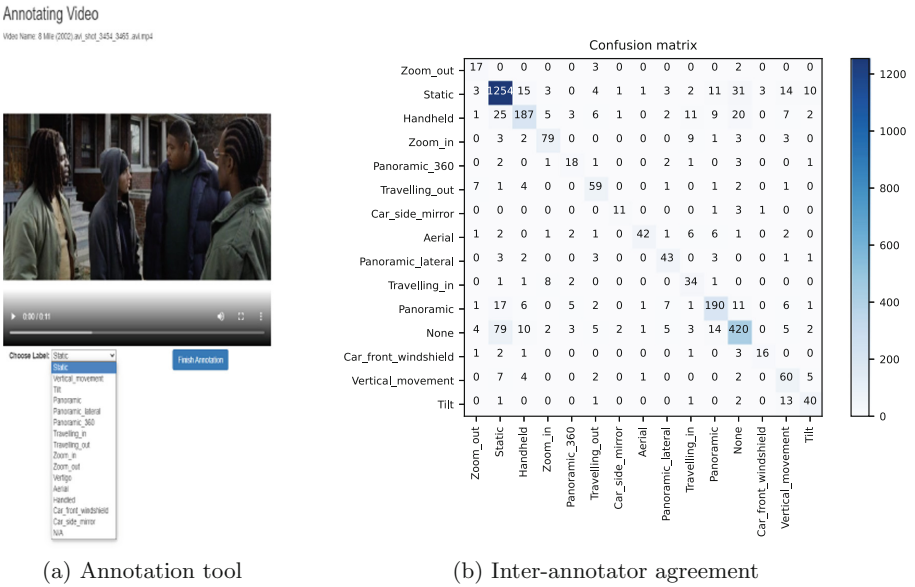


Fig. 1. Web tool and annotation agreement

This means that if a shot was annotated by two persons, this shot will be used in the final ground truth only if both annotators agreed on the label. Similarly, if three persons annotated a video shot, this would be used in the final ground truth if at least two of the annotators agreed on the label (higher than 60% agreement). Figure 1(b) visualizes the agreement between the individual annotators for all adopted classes. The overall inter-annotator agreement was above 80%.

After this process, 1877 videos from all 15 classes “survived” with a confident aggregated label. However, for 5 of these classes, the number of samples was below 30 and therefore have not been used in the final dataset. Given that, the final dataset size was 1803. Table 1 shows the final aggregated counts per final class (for the 10 adopted classes).

3 Video Shot Classification Methods

3.1 Feature Extraction

Image classification methods are usually based on features extracted from well-established Deep Learning computer vision architectures, such as the VGG [28] and ResNet [14] networks. However, in the task of Video Shot Classification we are more interested in capturing the image flow rather than the actual visual information.

Table 1. Number of samples per class of the final dataset

Class	Samples
Static	985
Handheld	273
Panoramic	207
Travelling in	55
Vertical movement	52
Aerial	51
Zoom in	51
Travelling out	46
Panoramic lateral	46
Tilt	37

That is the reason why the hand-crafted video features presented in [22], which meaningfully describe the flow of the visual information, were chosen. In particular, every 0.2 s, the following 88 visual features have been extracted from the corresponding frame:

- Color - related features (45 features):
 - 8-bin histogram of the red values
 - 8-bin histogram of the green values
 - 8-bin histogram of the blue values
 - 8-bin histogram of the grayscale values
 - 5-bin histogram of the max-by-mean-ratio for each RGB triplet
 - 8-bin histogram of the saturation values
- Average absolute difference between two successive frames in grey scale (1 feature)
- Facial features (2 features): The Viola-Jones [32] OpenCV implementation is used to detect frontal faces and the following features are extracted per frame:
 - number of faces detected
 - average ratio of the faces' bounding boxes areas divided by the total area of the frame
- Optical-flow related features (3 features): The optical flow is estimated using the Lucas-Kanade method [19] and the following 3 features are extracted:
 - average magnitude of the flow vectors
 - standard deviation of the angles of the flow vectors
 - a hand-crafted feature that measures the possibility that there is a camera tilt movement – this is achieved by measuring a ratio of the magnitude of the flow vectors by the deviation of the angles of the flow vectors.
- Current shot duration (1 feature): a basic shot detection method is implemented in this library. The length of the shot (in seconds) in which each frame belongs to, is used as a feature.

- Object-related features (36 features): We use the Single Shot Multibox Detector [18] method for detecting 12 categories of objects. For each frame, as soon as the object(s) of each category are detected, three statistics are extracted: number of objects detected, average detection confidence and average ratio of the objects’ area to the area of the frame. So in total, $3 \times 12 = 36$ object-related features are extracted. The 12 object categories we detect are the following: person, vehicle, outdoor, animal, accessory, sports, kitchen, food, furniture, electronic, appliance and indoor.

For extracting these visual features, the *multimodal_movie_analysis* library³ has been used in order to obtain features representing visual characteristics of a video.

The aforementioned features provide a wide range of low (simple color aggregates), mid (optical flows) and high (existence of objects and faces) representation levels. The rationale behind the selection of this wide range of types of features lies in the fact that our goal is to cover a great variety of flow information that may possibly be correlated to the camera movements.

In order to approach the problem from two different perspectives, we create two different types of features:

– Static features

The following video-level statistics are calculated for each frame sequence:

- six (6) video-level statistics of the non-object features. In particular, mean, standard deviation (std), median by std ratio, top-10 percentile, mean of the delta features and std of the delta features
- for the object detection, the frame-level predictions are post processed under local time windows with two different ways: (i) the object frame-level confidences are smoothed across time windows in order to increase the accuracy of the predictions and (ii) every object that is not present to at least a minimum number (threshold) of subsequent frames, is excluded from the final feature vector. However, this smoothing procedure is the only post-processing performed on the object-related features: no other statistics are extracted for the whole video, other than the object features’ simple averages.

This process therefore results to $52 \times 6 + 36 = 348$ feature statistics that describe the whole video in a static feature vector.

– Sequential features

A post processing procedure is applied on the feature sequence in order to smooth the representation across successive frames. Median-filtering with kernel size of 4 (i.e. 0.8s) were chosen. The processed sequence can be used in a temporal modeling.

3.2 Baseline Classification

In this section we propose two distinct classification methods that can give an intuition about the separability of the introduced classes. These methods will

³ https://github.com/tyiannak/multimodal_movie_analysis.

serve in order to report baseline classification metrics and understand the latent correlations across classes.

Static Method. The first method is based on the static video representation described in Sect. 3.1. It adopts an SVM algorithm with the appropriate data normalization and parameter tuning. This approach aims on finding significant correlations between statistical frame representations and the target shot styles. Such methods can learn to separate continuous camera movements from static shots, but may under-perform in cases of abrupt shot style changes.

Sequential Method. This method is aimed to predict the shot style from the temporal feature sequence. In such a case it would be easier to model different kinds of camera movements since the approach is expected to capture the temporal frame dependencies.

An LSTM architecture was chosen, since it is a well established Recurrent Neural Network that can capture long-term temporal information [11]. Batch-normalization [15] and linear layers were used along with ReLU activation functions in order to perform the classification task. Temporal standard scaling was applied to the input sequence, that is each instance of a feature along the sequence were standardized using the same statistical values (mean value and standard deviation).

4 Experiments

In this section, experimental results are presented for both static and sequential methods.

4.1 Classification Tasks

By combining different shot categories, four classification tasks, one binary and three multi-label, are defined.

Binary Classification Task. The binary task includes the *static* and *non-static* classes. The former consists of shots that have been annotated as **static**, while the latter contains all the classes from the original dataset that are associated with any type of camera movement. That is the corresponding sub-classes are **Panoramic Lateral, Vertical Movement, Zoom-in, Handheld, Aerial, Tilt, Panoramic, Travelling-in, Travelling-out**.

Multi-label Classification Tasks. As it will be described in the results section, predicting the type of camera movement is not a trivial task. At the same time, as explained in the introduction, it is a task that can be mostly used as an automated attribute movie extractor, that is used in the context of a more

complex system (such as movie recommendation). Therefore, it is not a task that can form a standalone application or product. Given that, it makes sense to analyze the movie in various levels of detail with regards to its camera movement styles. In this paper, apart from the initial 10-class classification task, we have defined two more classification tasks, by merging classes of similar camera movement (e.g. all vertical movements).

So in total, we define three multi-label classification tasks in which the static class is the original annotated class, while the others are merges from other class combinations.

- **3-class** The corresponding classes are *Zoom*, *Static* and *Vertical & Horizontal Movements*. The **Zoom** class consists of the *Zoom-in*, *Travelling-in* and *Travelling-out* sub-classes, which all contain shots in which the perimeter image changes at very fast intervals, while the centre image remains static or changes at a slower rate. The **Vertical & Horizontal Movements** class consists of the *Vertical Movement*, *Tilt*, *Panoramic* and *Panoramic Lateral* sub-classes from the original dataset, where the position of the camera is moving either in a vertical or in a horizontal way.
- **4-class** In this task, the **Static** and **Zoom** classes of the 3-class problem were kept, while the Vertical & Horizontal Movements class was separated into 2 sub-classes, **Tilt**, which includes all vertical movements and consists of the *Vertical Movement* and *Tilt* original classes, and **Panoramic** that contains shots with lateral movements and consists of the *Panoramic* and *Panoramic Lateral* original classes.
- **10-class** This task includes all provided class from the original dataset, namely: **Static**, **Panoramic**, **Zoom-in**, **Travelling-out**, **Vertical Movement**, **Aerial**, **Travelling-in**, **Tilt**, **Handheld** and **Panoramic Lateral**.

4.2 Experimental Set-Up

With regard to the *static* method, an SVM classifier with rbf kernel were initialized. After the data has been scaled using a zero-mean normalization, parameter tuning was performed over the ‘C’ parameter.

Concerning the *sequential* method, we initially performed a sequential zero-mean normalization. That is, by treating each frame as a different instance, the mean and standard deviation of all features across all frames of the training set of the dataset was calculated and then used to normalize each frame of the dataset, resulting in normalized feature sequences. The LSTM architecture was tuned for each of the four tasks in order to achieve the optimal dropout, weight decay, learning rate and LSTM’s hidden dimension. For the binary task, the binary cross entropy loss was used, while for the rest, cross entropy was adopted. Adam was chosen to be the optimizer with the integration of a reduce-on-plateau learning rate scheduler, while the best model was chosen using early stopping.

In order to evaluate the classifiers a cross validation procedure of totally 10 iterations was performed. For each iteration, the shots of the dataset were

randomly split in a stratified manner (i.e. keep the initial class distribution) into 20% test set for testing, 9.6% validation set for parameter tuning and 70.4% train set for training.

4.3 Results

Table 2. Macro-averaged F1 of (a) all classes and (b) the non-neutral classes (i.e. excluding static class) calculated on the aggregated predictions across all 10 cross-validated iterations

(a)			(b)		
Classification task	Static	Sequential	Classification task	Static	Sequential
2-class	73.9%	79%	3-class	35.7%	39.8%
3-class	50.6%	53%	4-class	24.8%	27.8%
4-class	39.5%	40%			
10-class	15.7%	15.4%			

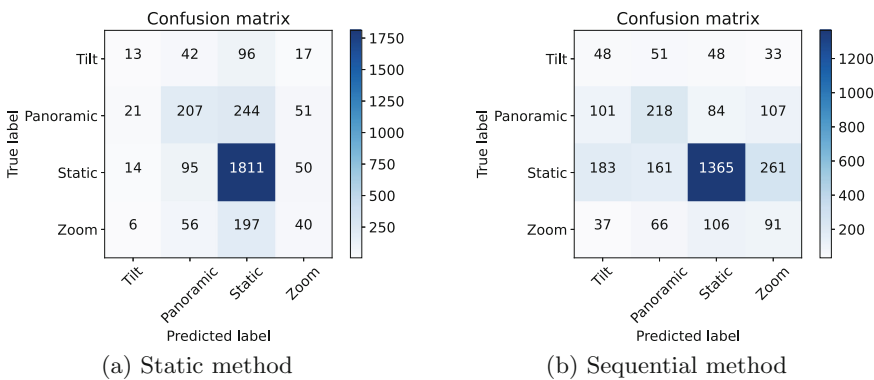


Fig. 2. Confusion matrices based on aggregated predictions for the 4-class task

The resulted evaluation metrics are listed in Table 2, while the corresponding, aggregated from all iterations, confusion matrices are presented in Fig. 2 for the 3-class and 4-class problems.

As can be clearly seen from the results, the sequential method completely outperforms the static one in terms of the binary task, by scoring a 7% relatively higher score. That is an expected result since the basic distinction of the static and non-static classes is the camera movement, which is easier to capture by a sequential method. With regard to 3-class and 4-class problems, the sequential method achieves a higher by 4.7% and 1.3% relative performance respectively, while for the 10-class task the static method is 1.9% relatively better. It is

observed that the more the classes are in the tasks, the less the difference between the two methods is. That is mostly due to the fact that some of the 10 classes are underrepresented and thus a deep learning architecture is harder to learn from that amount of data compared to a traditional machine learning algorithm.

However, in such tasks where the neutral class (static class) is dominant in terms of number of instances, the macro-averaged F1 metric cannot properly represent the performance of the methods. By examining the confusion matrices of Fig. 2, it can be clearly seen that the sequential method achieves to adequately predict the non-neutral classes (tilt, panoramic and zoom). More specifically, as illustrated in Table 2b, which includes the macro-averaged F1 metrics based only on the non-neutral classes (i.e. excluding the static class), the sequential method is 11.5% and 12.1% relatively better for the 3 and 4-class problem respectively. Thus it can be inferred that including temporal information in the classifier leads to better distinction across camera movements.

It needs to be noted that the selection of the features used in our method was not random. More specifically, our pipeline was evaluated by training and testing the LSTM on feature sequences extracted from the first linear layer of the pre-trained VGG16 network. The results showed that when the LSTM is trained on our features (produced by the *multimodal_movie_analysis* library) it performs 21.5%, 23.6%, 24% and 7.7% better for the 2, 3, 4 and 10-class tasks respectively, compared to the model trained on the VGG16 features. This outcome is reasonable since while the VGG features contain better image-related information, our features achieve to better model the camera movement, an attribute that is crucial for shot type classification.

5 Conclusion and Future Work

In this work, a new publicly available⁴ dataset for film shot classification with annotations in the camera movement styles was presented. This data collection fills the gap in the literature of cinematographic style analysis, since the existing datasets are either private or are not concerned about camera movements in an extensive manner. Additionally, two different methods were used as baseline evaluation, one static and one sequential showing that camera movement classification is better performed when modeling sequential information. Models trained on this dataset can be used in a future work in order to create artistic profiles of either directors or movies that can describe in an informative way the corresponding work. In addition, this can provide a new way of representing the movies in recommender systems, so that the extracted recommendations are content-driven and, in particular, driven by the actual underlying visual aesthetics of the movies.

⁴ https://github.com/magcil/movie_shot_classification_dataset.

References

1. Bak, H.Y., Park, S.B.: Comparative study of movie shot classification based on semantic segmentation. *Appl. Sci.* **10**(10), 3390 (2020). <https://doi.org/10.3390/app10103390>, <https://www.mdpi.com/2076-3417/10/10/3390>
2. Baraldi, L., Grana, C., Cucchiara, R.: Shot and scene detection via hierarchical clustering for re-using broadcast video. In: Azzopardi, G., Petkov, N. (eds.) *CAIP* 2015. *LNC3*, vol. 9256, pp. 801–811. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23192-1_67
3. Benini, S., Svanera, M., Adami, N., Leonardi, R., Kovács, A.B.: Shot scale distribution in art films. *Multimedia Tools Appl.* **75**(23), 16499–16527 (2016). <https://doi.org/10.1007/s11042-016-3339-9>
4. Bhattacharya, S., Mehran, R., Sukthankar, R., Shah, M.: Classification of cinematographic shots using lie algebra and its application to complex event recognition. *IEEE Trans. Multimedia* **16**(3), 686–696 (2014)
5. Bougiatiotis, K., Giannakopoulos, T.: Enhanced movie content similarity based on textual, auditory and visual information. *Expert Syst. Appl.* **96**, 86–102 (2018)
6. Braudy, L.: Film: an international history of the medium. *Film Q.* (ARCHIVE) **48**(3), 59 (1995)
7. Canini, L., Benini, S., Leonardi, R.: Classifying cinematographic shot types. *Multimedia Tools Appl.* **62**(1), 51–73 (2013)
8. Choi, S.M., Ko, S.K., Han, Y.S.: A movie recommendation algorithm based on genre correlations. *Expert Syst. Appl.* **39**(9), 8079–8085 (2012)
9. Diao, Q., Qiu, M., Wu, C.Y., Smola, A.J., Jiang, J., Wang, C.: Jointly modeling aspects, ratings and sentiments for movie recommendation (JMARS). In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 193–202 (2014)
10. Ertugrul, A.M., Karagoz, P.: Movie genre classification from plot summaries using bidirectional LSTM. In: *2018 IEEE 12th International Conference on Semantic Computing (ICSC)*, pp. 248–251. IEEE (2018)
11. Greff, K., Srivastava, R.K., Koutník, J., Steunebrink, B.R., Schmidhuber, J.: LSTM: a search space odyssey. *IEEE Trans. Neural Networks Learn. Syst.* **28**(10), 2222–2232 (2016)
12. Haq, I.U., Muhammad, K., Hussain, T., Kwon, S., Sodanil, M., Baik, S.W., Lee, M.Y.: Movie scene segmentation using object detection and set theory. *Int. J. Distrib. Sens. Networks* **15**(6), 1550147719845277 (2019)
13. Hasan, M.A., Xu, M., He, X., Xu, C.: CAMHID: camera motion histogram descriptor and its application to cinematographic shot classification. *IEEE Trans. Circ. Syst. Video Technol.* **24**(10), 1682–1695 (2014). <https://doi.org/10.1109/TCSVT.2014.2345933>
14. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016)
15. Ioffe, S., Szegedy, C.: Batch normalization: accelerating deep network training by reducing internal covariate shift. In: *International Conference on Machine Learning*, pp. 448–456. PMLR (2015)
16. Lekakos, G., Caravelas, P.: A hybrid approach for movie recommendation. *Multimedia Tools Appl.* **36**(1), 55–70 (2008)
17. Li, K., Li, S., Oh, S., Fu, Y.: Videography-based unconstrained video analysis. *IEEE Trans. Image Process.* **26**(5), 2261–2273 (2017). <https://doi.org/10.1109/TIP.2017.2678800>

18. Liu, W., et al.: SSD: single shot multibox detector. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds.) ECCV 2016. LNCS, vol. 9905, pp. 21–37. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46448-0_2
19. Lucas, B.D., Kanade, T., et al.: An iterative image registration technique with an application to stereo vision. Vancouver, British Columbia (1981)
20. Minhas, R.A., Javed, A., Irtaza, A., Mahmood, M.T., Joo, Y.B.: Shot classification of field sports videos using AlexNet convolutional neural network. *Appl. Sci.* **9**(3), 483 (2019)
21. Park, S.C., Lee, H.S., Lee, S.W.: Qualitative estimation of camera motion parameters from the linear composition of optical flow. *Pattern Recogn.* **37**(4), 767–779 (2004)
22. Psallidas, T., Koromilas, P., Giannakopoulos, T., Spyrou, E.: Multimodal summarization of user-generated videos. *Appl. Sci.* **11**(11), 5260 (2021). <https://doi.org/10.3390/app11115260>, <https://www.mdpi.com/2076-3417/11/11/5260>
23. Rao, A., et al.: A unified framework for shot type classification based on subject centric lens. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.-M. (eds.) ECCV 2020. LNCS, vol. 12356, pp. 17–34. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58621-8_2
24. Rasheed, Z., Shah, M.: Scene detection in Hollywood movies and tv shows. In: 2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. Proceedings, vol. 2, p. II-343. IEEE (2003)
25. Sang, J., Xu, C.: Character-based movie summarization. In: Proceedings of the 18th ACM International Conference on Multimedia, pp. 855–858 (2010)
26. Savardi, M., Signoroni, A., Migliorati, P., Benini, S.: Shot scale analysis in movies by convolutional neural networks. In: 2018 25th IEEE International Conference on Image Processing (ICIP), pp. 2620–2624. IEEE (2018)
27. Simões, G.S., Wehrmann, J., Barros, R.C., Ruiz, D.D.: Movie genre classification with convolutional neural networks. In: 2016 International Joint Conference on Neural Networks (IJCNN), pp. 259–266. IEEE (2016)
28. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (2014)
29. Subramaniaswamy, V., Logesh, R., Chandrashekhar, M., Challa, A., Vijayakumar, V.: A personalised movie recommendation system based on collaborative filtering. *Int. J. High Perform. Comput. Networking* **10**(1–2), 54–63 (2017)
30. Tsai, C.M., Kang, L.W., Lin, C.W., Lin, W.: Scene-based movie summarization via role-community networks. *IEEE Trans. Circ. Syst. Video Technol.* **23**(11), 1927–1940 (2013)
31. Ul Haq, I., Ullah, A., Muhammad, K., Lee, M.Y., Baik, S.W.: Personalized movie summarization using deep CNN-assisted facial expression recognition. In: Complexity 2019 (2019)
32. Viola, P., Jones, M.: Rapid object detection using a boosted cascade of simple features. In: Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001, vol. 1, p. I-I. IEEE (2001)
33. Wang, H.L., Cheong, L.F.: Taxonomy of directing semantics for film shot classification. *IEEE Trans. Circ. Syst. Video Technol.* **19**(10), 1529–1542 (2009). <https://doi.org/10.1109/TCSVT.2009.2022705>
34. Zhou, H., Hermans, T., Karandikar, A.V., Rehg, J.M.: Movie genre classification via scene categorization. In: Proceedings of the 18th ACM International Conference on Multimedia, pp. 747–750 (2010)